Curso: Ciências da Computação

Disciplina: Laboratório de Programação Orientada a Objetos

Aluno: Marcos Vinicius da Silva Alves

Professor: Paulo Nunes

## Bloco A

**Questão 1 - Explique a diferença entre classe e objeto. Apresente um exemplo de cada um em um sistema acadêmico.**

Uma classe é um modelo que define as caracteristicas e os comportamentos de algo, já um objeto é uma instância criada através desse modelo. Por exemplo, uma classe Aluno define os dados de um estudante, enquanto o objeto representa um aluno especifico.

**Questão 2 - O que são atributos e métodos? Diferencie estado e comportamento de um objeto.**

Atributos são variáveis que armazenam as caracteristicas de um objeto, já os métodos representam suas ações. O estado corresponde aos valores de um atributo em determinado momento, enquanto o comportamento define o que o objeto pode fazer por meio dos métodos.

**Questão 3 - Qual é a função de new? Diferencie criar um objeto e declarar uma referência.**

A palavra chave `new` cria um objeto na memória e chama seu construtor. Declarar uma referência significa criar uma variável capaz de apontar para esse objeto sem a necessariamente criar um objeto naquele momento.

**Questão 4 - Explique encapsulamento. Por que evitar atributos públicos sem necessidade?**

O encapsulamento é um princípio da programação orientada a objetos que protege os dados de uma classe, controlando o acesso a seus atributos por meio de métodos, como getters e setters. Devemos evitar atributos públicos sem necessidade pois ele podem ser modificados diretamente sem o uso de validações, o que o pode comprometer a segurança e a integridade das informações.

**Questão 5 - Qual é a finalidade do construtor? Explique o uso de this.**

O construtor é criado durante a criação de um objeto e serve para inicializar seus atributos. A palavra chave `this` referência o proprio objeto,sendo usada, por exemplo, para diferenciar um atributo de um paraâmetro de mesmo nome.

**Bloco B**

**Questão 6 - Diferencie associação, agregação e composição; exemplifique.**

Na associação um objeto se relaciona/conhece o outro, mas eles existem de forma independente, por exemplo Livro e Autor, o livro possui um autor, mas um autor pode existir sem aquele livro.

Na agregação o objeto possui ou reúne vários outros objetos, mas ele pode existir de forma independente, como a Biblioteca que possui Livros, entretanto se a biblioteca deixa de existir os livros ainda existirão separadamente.

E por fim na composição, um objeto é formado por outros objetos que são suas partes, com uma relação mais forte de dependência. No caso de Livro e Exemplar, o Exemplar representa uma cópia específica do Livro e seu ciclo de vida está ligado ao Livro dentro do modelo.

**Questão 7 - O que é ArrayList e quando preferi-lo a um vetor de tamanho fixo?**

Um `ArrayList` é uma estrutura de coleção que armazena elementos e permite aumentar ou diminuir o tamanho conforme o necessario. É preferivel a um vetor de tamanho fixo quando a quantidade de elementos varia durante a execução.

**Questão 8 - Diferencie superclasse e subclasse. Explique extends e super**.

A superclasse é a classe mais geral que fornece atributos e métodos. A subclasse herda as caracteristicas da superclasse e pode adicionar seus proprios métodos e atributos. O `extends` estabelece uma herança entre classes, já `super` permite acessar membros da superclasse ou chamar seu construtor.

**Questão 9 - Diferencie sobrecarga e sobrescrita de métodos; dê exemplos.**

A sobrecarga (overloading) ocorre quando métodos possuem o mesmo nome, mas parâmetros diferentes. Exemplo de sobrecarga:

```
    void exibir(String nome){}
    void exibir(int matricula){
```

Já a sobrescrita (overriding) ocorre quando uma subclasse redefine um método herdado. Exemplo de sobrescrita :

```
class pessoa {
  void apresentar(){
   System.out.println("Pessoa!");
  }
}

class Aluno extends Pessoa{
  @Override
  void apresentar(){
   System.out.println("Aluno!");
  }
}
```

**Questão 10 - O que é polimorfismo? Como uma referência de superclasse aponta para objetos de subclasses?**

Polimorfismo é a capacidade de utilizar objetos de diferentes subclasses por meio de uma referência do tipo superclasse. A referência pode apontar para objetos de subclasses diferentes, e o método executado será definido de acordo com o tipo real do objeto. Por exemplo, uma referência Animal pode apontar para um objeto Cachorro ou Gato, executando o método emitirSom() de cada um

**Bloco C**

**Questão 11 - O que é interface? Diferencie implements e extends nos exemplos estudados.**

Uma interface define um contrato de métodos que uma classe deve implementar. O `implements` é usado quando uma classe implementa uma interface, enquanto o `extends` é usado para herdar uma classe.

**Questão 12 - Compare herança e composição. Quando a composição é preferível?**

Herança representa uma relação de "é um", como Aluno sendo uma Pessoa. Composição representa uma relação de "tem um", como Aluno tendo um endereço.

A Composição é preferivel quando se deseja reutilizar funcionalidades sem criar uma dependência rigida entre classes, facilitando alterações e manutenções.

**Questão 13 - O que é baixo acoplamento? Como a injeção por construtor ajuda?**

Baixo acoplamento é quando as classes possuem poucas dependências entre si, facilitando alterações e manutenção do código. A injeção por construtor ajuda porque permite que as dependências sejam fornecidas à classe por meio do construtor, em vez de serem criadas diretamente nela, tornando o código mais flexível, reutilizável e fácil de testar.

**Questão 14 - Explique SRP, OCP e DIP com exemplos simples.**

SRP(Princípio da Responsabilidade Única): Uma classe deve ter um motivo principal para mudar. Exemplo: uma classe calcula notas e outra gera relatorios.

OCP(Princípio Aberto/Fechado): O sistema deve permitir novas funcionalidades sem exigir alterações constantes no código existente. Exemplo: adicionar novos tipos de desconto por meio de novas classes.

DIP(Princípio da Inversão de Dependência): Módulos devem depender de abstrações, não diretamente de implementações concretas. Exemplo: um serviço de mátricula depende de uma interface de pagamento, e não de uma classe especifica.

**Bloco D**

**Questão 15 - O que é um padrão de projeto? Por que evitar aplicá-lo indiscriminadamente?**

Um padrão de projeto é uma solução reutilizável para um problema recorrente na organizaçao do software. Deve ser utilizado quando realmente ajuda, pois sua aplicação desnecessarias pode aumentar a complexidade e dificultar a manutenção.

**Questão 16 - Explique Singleton e a finalidade do construtor privado e de getInstancia().**

Singleton é um padrão de projeto que garante que uma classe tenha apenas uma instância. O construtor privado impede que outros objetos criem instâncias diretamente, enquanto o método `getInstancia()` permite acessar a única instância criada.

**Questão 17 - Explique Factory Method e o papel dos criadores abstratos e concretos.**

O Factory Method é um padrão de projeto que permite criar objetos sem depender diretamente de classes específicas. O criador abstrato define o método de criação, enquanto os criadores concretos implementam esse método para criar objetos específicos.

**Questão 18 - Como Strategy organiza descontos comuns, estudantis e promocionais?**

O padrão Strategy permite criar diferentes algoritmos de desconto por meio de uma interface comum. Cada estratégia implementa uma regra específica, como desconto comum, estudantil ou promocional. Assim, o sistema pode selecionar a estratégia adequada em tempo de execução sem modificar sua estrutura principal, facilitando a manutenção e a adição de novas regras de desconto.

**Questão 19 - Como Observer comunica um pedido confirmado a estoque, financeiro e auditoria?**

O padrão Observer permite que vários componentes sejam notificados quando ocorre um evento. Quando um pedido é confirmado, o sistema notifica os observadores registrados, como estoque, financeiro e auditoria, para que cada um execute sua tarefa específica. Isso reduz o acoplamento, pois o pedido não precisa conhecer os detalhes internos de cada componente notificado.

**Questão 20 - Diferencie Adapter e Facade; dê um caso de uso de cada um.**

O Adapter permite que sistemas com interfaces incompatíveis trabalhem juntos, adaptando uma interface para outra. Um exemplo é integrar um sistema antigo de pagamento a uma nova aplicação.

O Facade simplifica o acesso a um sistema complexo, oferecendo uma interface única para vários componentes. Um exemplo é criar uma função para finalizar uma compra, que coordena o pagamento, o estoque e a emissão da nota fiscal.